Name : Jithin Jyothi

Roll : AM.SC.U4CSE23042

Prompts used :

1. Finding the repo based on the task

Prompt :

I am doing a university case study titled “Design Pattern Identification and Evolution in a Spring Boot-Based E-Commerce Application.” Find an open-source e-commerce application developed using Spring Boot. It should have accessible source code and features such as product management, cart, orders, payments, inventory, customers, notifications, and authentication. Compare suitable repositories against these requirements and recommend the best one. Support the recommendation with repository links and source evidence.

2. Checking if the part 2 changes to make already exist in the project.

Prompt :

Inspect the supplied Shopizer source before I modify it. The repository is https://github.com/shopizer-ecommerce/shopizer.git, using tag/version label 3.2.7. Check whether these requirements are already implemented:

1. Add Razorpay as a payment method.

2. Require category-specific product attributes, such as a warranty for Electronics.

3. Send SMS notifications when an order's status changes.

3. Making the part 1 report.

Prompt :

I have attached my Part 1 report draft, the required template, and the shopizer zip. Correct the report so that the application description, architecture, design patterns, class names, source paths, tables, and evidence match the actual project code. Follow the provided template and preserve correct content. Do not invent missing details or state a pattern not present. Give the correct document.

4. Making the part 2 report.

Prompt ：

Part 1 is complete, now help me with part 2 using this template, findings, and Shopizer zip. The three changes are Razorpay payment integration, mandatory Electronics product-attribute validation, and SMS notification for order-status changes. For each, explain the requirement, existing design, proposed change, affected components, and impact on relevant patterns. Classify each pattern as preserved, modified, extended, replaced, or removed, with a reason supported by code evidence. Continue page numbering after page 8 and do not repeat part 1.

5. Choosing the two patterns for part 3.
   Part 3 – Cross-Language Analysis
   Select 2 significant patterns discovered in Parts 1–2.
   Implement the same problem using:
   - Java
   - Python
   - JavaScript
   - C++

Proceeded with the recommended pattern.

6. Starting the implementation of Observer and Strategy using Python.

   Prompt ：

   Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

7. After confirming the cases of Python, started with JavaScript implementation.

   Prompt :

   Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the

behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.

8. After JS implementation, moved to Java implementation.

   Prompt :

   After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.

9. After JS implementation, moved to C++ implementation.

   Prompt :

   Now, implement the same payment strategy and order status observer problems in C++. Also provide the commands to run it.

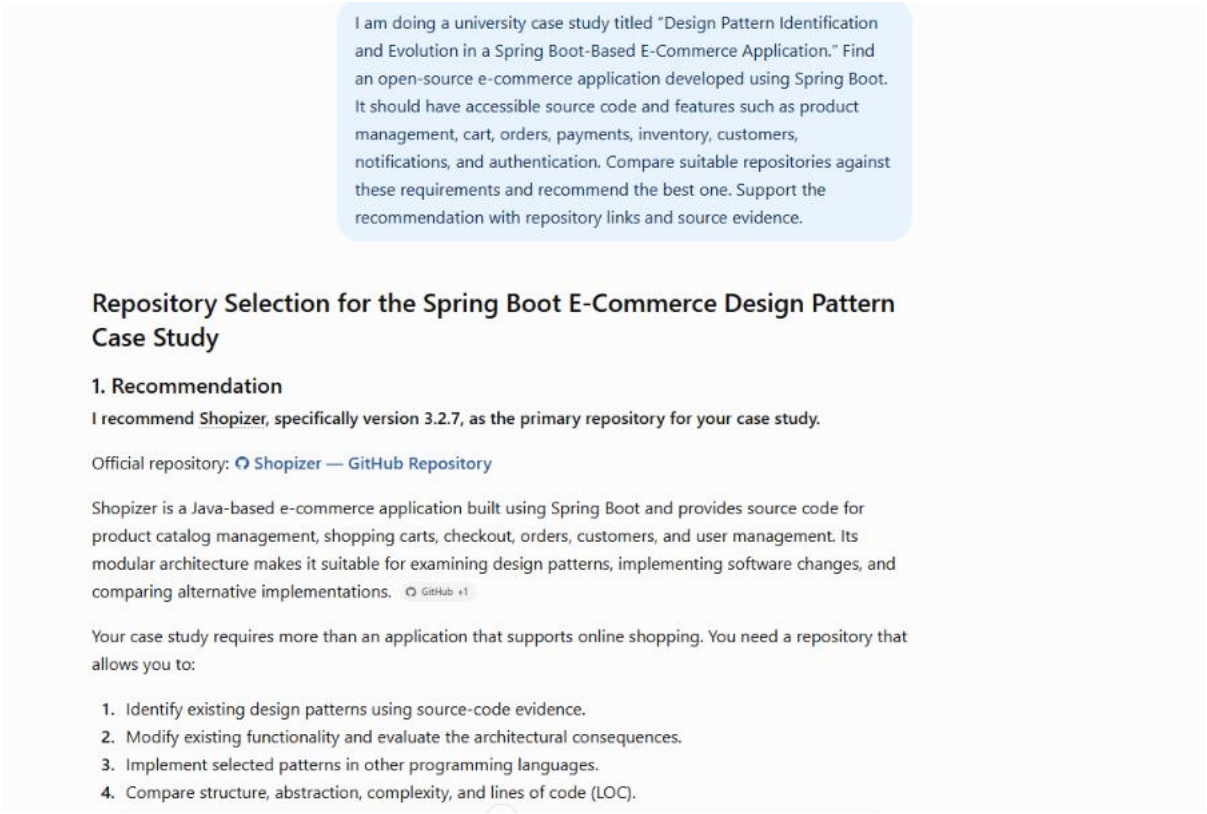
10. Verifying all four language implementation.

    Prompt :

    Inspect the final implementation zip and audit both strategy payment module selection and observer order status SMS notification across Java, Python, JavaScript and C++. Check all the documents and verify the correctness.

LLM outputs :

1.



2. Checking if the part 2 changes to make already exist in the project.

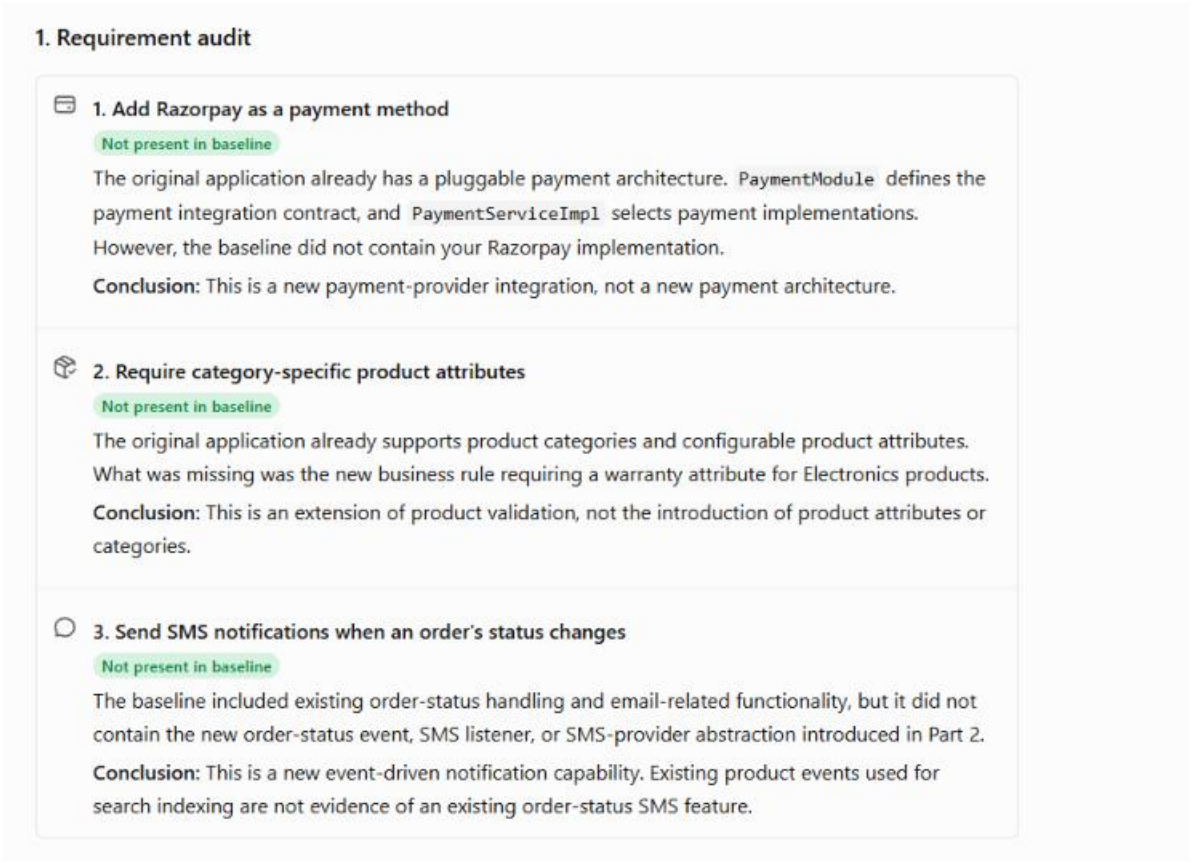
Prompt ：

Inspect the supplied Shopizer source before I modify it. The repository is https://github.com/shopizer-ecommerce/shopizer.git, using tag/version label 3.2.7. Check whether these requirements are already implemented:

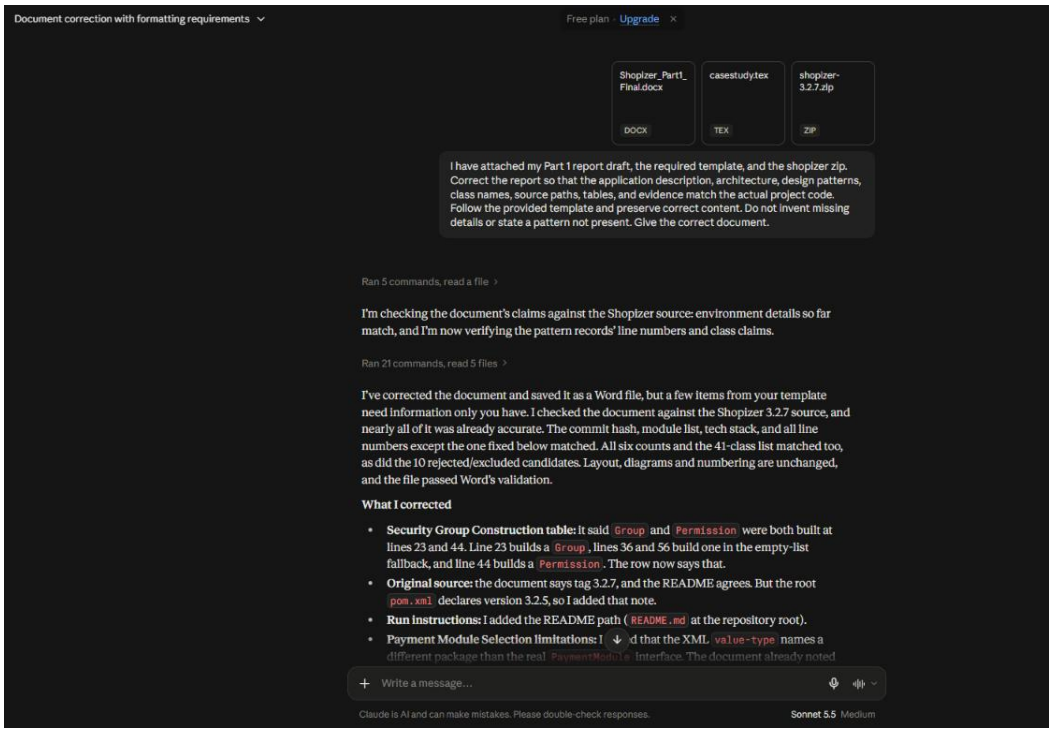
1. Add Razorpay as a payment method.

2. Require category-specific product attributes, such as a warranty for Electronics.

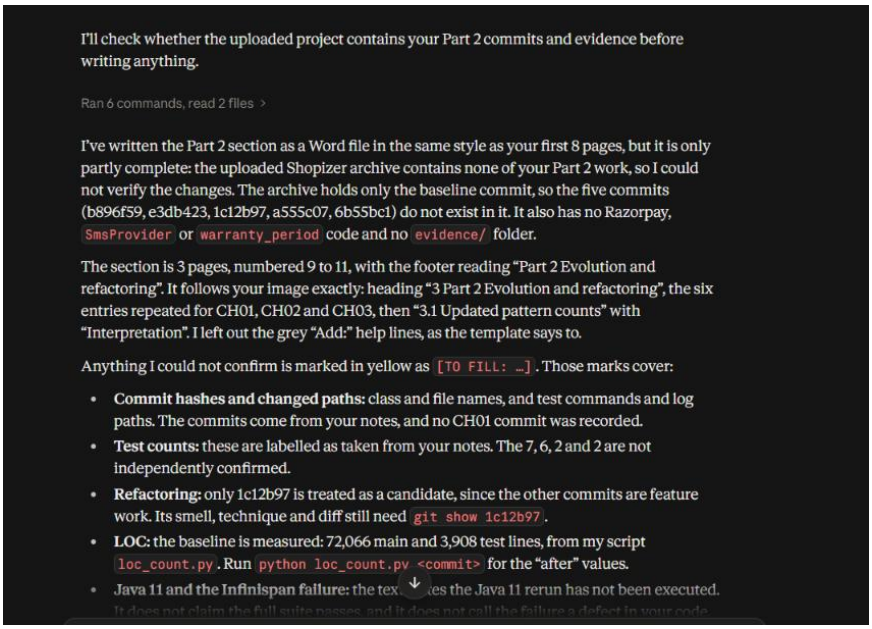
3. Send SMS notifications when an order's status changes.



3. I have attached my Part 1 report draft, the required template, and the shopizer zip. Correct the report so that the application description, architecture, design patterns, class names, source paths, tables, and evidence match the actual project code. Follow the provided template and preserve correct content. Do not invent missing details or state a pattern not present. Give the correct document.



4. Part 1 is complete, now help me with part 2 using this template, findings, and Shopizer zip. The three changes are Razorpay payment integration, mandatory Electronics product-attribute validation, and SMS notification for order-status changes. For each, explain the requirement, existing design, proposed change, affected components, and impact on relevant patterns. Classify each pattern as preserved, modified, extended, replaced, or removed, with a reason supported by code evidence. Continue page numbering after page 8 and do not repeat part 1.



I'll check whether the uploaded project contains your Part 2 commits and evidence before writing anything.

Ran 6 commands, read 2 files >

I've written the Part 2 section as a Word file in the same style as your first 8 pages, but it is only partly complete: the uploaded Shopizer archive contains none of your Part 2 work, so I could not verify the changes. The archive holds only the baseline commit, so the five commits (b896f59, e3db423, 1c12b97, a555c07, 6b55bc1) do not exist in it. It also has no Razorpay, `SmsProvider` or `warranty_period` code and no `evidence/` folder.

The section is 3 pages, numbered 9 to 11, with the footer reading "Part 2 Evolution and refactoring". It follows your image exactly: heading "3 Part 2 Evolution and refactoring", the six entries repeated for CH01, CH02 and CH03, then "3.1 Updated pattern counts" with "Interpretation". I left out the grey "Add:" help lines, as the template says to.

Anything I could not confirm is marked in yellow as `[TO FILL: …]`. Those marks cover:

- **Commit hashes and changed paths:** class and file names, and test commands and log paths. The commits come from your notes, and no CH01 commit was recorded.
- **Test counts:** these are labelled as taken from your notes. The 7, 6, 2 and 2 are not independently confirmed.
- **Refactoring:** only 1c12b97 is treated as a candidate, since the other commits are feature work. Its smell, technique and diff still need `git show 1c12b97`.
- **LOC:** the baseline is measured: 72,066 main and 3,908 test lines, from my script `loc_count.py`. Run `python loc_count.py <commit>` for the "after" values.
- **Java 11 and the Infinispan failure:** the tex…es the Java 11 rerun has not been executed. It does not claim the full suite passes, and it does not call the failure a defect in your code.

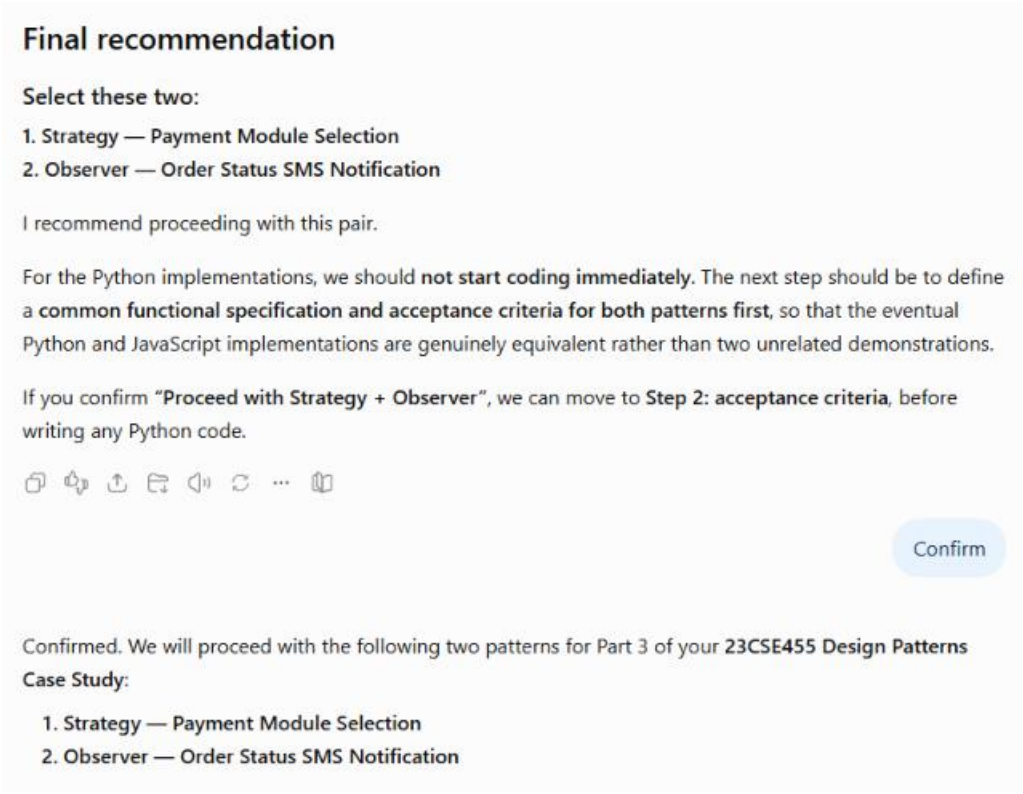
5. Proceeded with recommended pattern after reading through the comparisons provided.

Part 3 – Cross-Language Analysis

Select 2 significant patterns discovered in Parts 1–2.

Implement the same problem using:

- Java
- Python
- JavaScript
- C++



Final recommendation

Select these two:

1. Strategy — Payment Module Selection
2. Observer — Order Status SMS Notification

I recommend proceeding with this pair.

For the Python implementations, we should **not start coding immediately**. The next step should be to define a **common functional specification and acceptance criteria for both patterns first**, so that the eventual Python and JavaScript implementations are genuinely equivalent rather than two unrelated demonstrations.

If you confirm **"Proceed with Strategy + Observer"**, we can move to **Step 2: acceptance criteria**, before writing any Python code.

Confirm

Confirmed. We will proceed with the following two patterns for Part 3 of your **23CSE455 Design Patterns Case Study**:

1. **Strategy — Payment Module Selection**
2. **Observer — Order Status SMS Notification**

6. Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

Sunday 8:01 PM

DP_Csestudy(1).pdf
PDF

Lets start with the Python implementation in VSCode. First define the expected behavior and test cases for both patterns so that we can implement the same functionality in JavaScript, Java and C++ later.

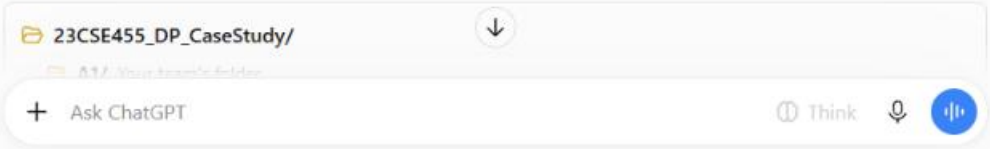
Part 3 — Initial Setup in VS Code

We will start with the Python implementation of **Strategy — Payment Module Selection**, then implement Observer after Strategy has been tested.

I have reviewed your assignment instructions. They specify a particular submission hierarchy, four language implementations (Java, Python, JavaScript and C++), separate production code and tests, and a CSV dataset for cross-language measurements. DP_Csestudy.pdf We should organize the project correctly from the beginning so that you do not need to restructure it before submission.

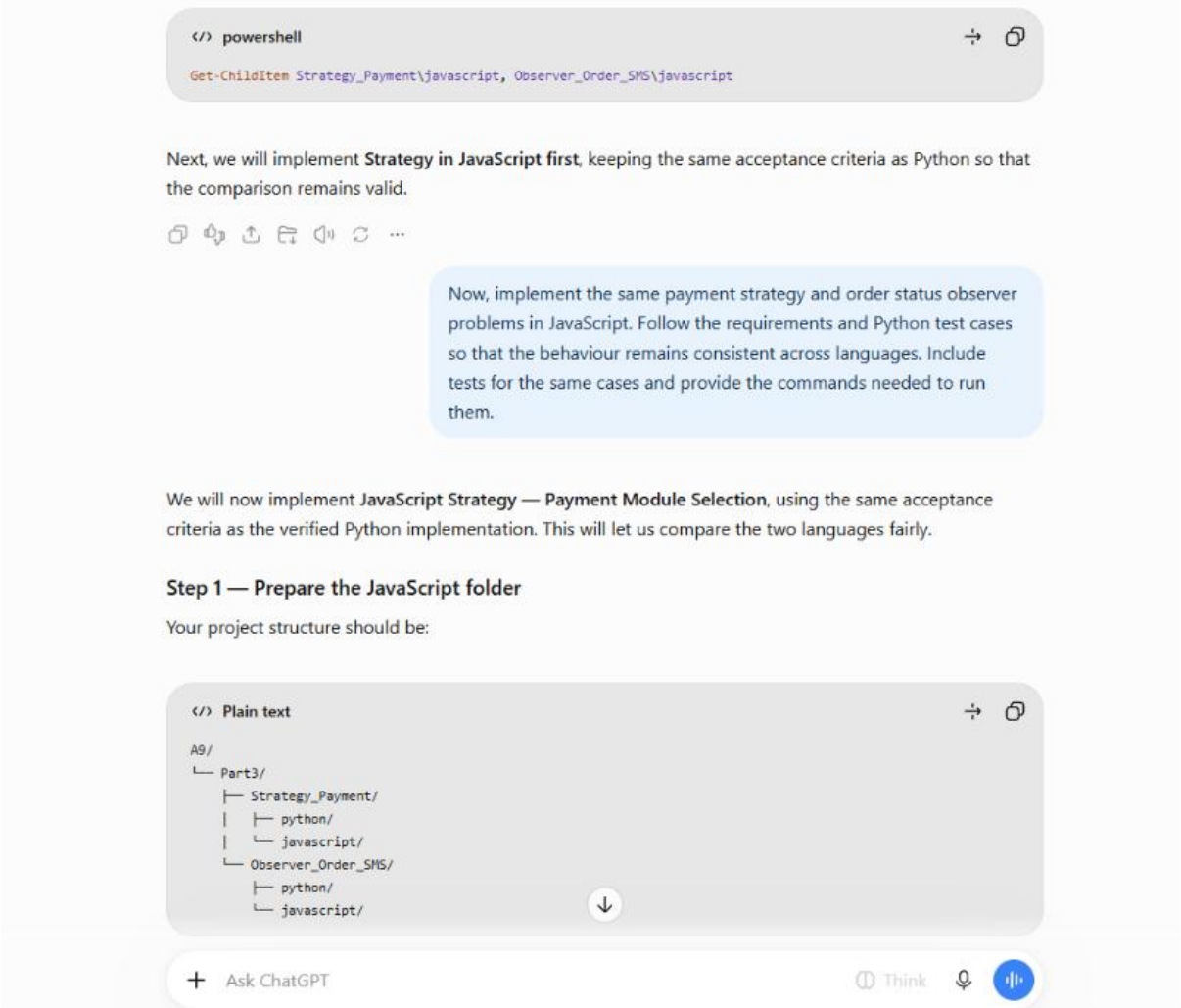
1. Understand where Part 3 belongs

Your assignment specifies the following hierarchy, using `A1` as an example team folder. Replace `A1` with your actual team folder name if different. DP_Csestudy.pdf



7.

Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.



```powershell
Get-ChildItem Strategy_Payment\javascript, Observer_Order_SMS\javascript
```

Next, we will implement **Strategy in JavaScript first**, keeping the same acceptance criteria as Python so that the comparison remains valid.

Now, implement the same payment strategy and order status observer problems in JavaScript. Follow the requirements and Python test cases so that the behaviour remains consistent across languages. Include tests for the same cases and provide the commands needed to run them.

We will now implement **JavaScript Strategy — Payment Module Selection**, using the same acceptance criteria as the verified Python implementation. This will let us compare the two languages fairly.

Step 1 — Prepare the JavaScript folder

Your project structure should be:

```
A9/
└── Part3/
    ├── Strategy_Payment/
    │   ├── python/
    │   └── javascript/
    └── Observer_Order_SMS/
        ├── python/
        └── javascript/
```

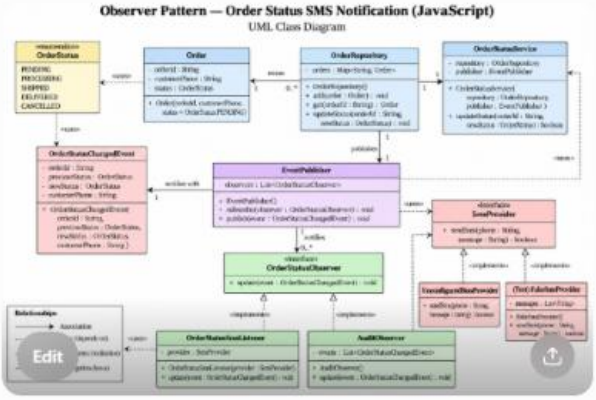
Ask ChatGPT Think

8. After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.

After pasting, press `Ctrl + S`.

Next task: create the JavaScript Observer UML diagram, then begin the Java implementations. Keep the recorded test results and LOC values tied to the actual terminal evidence you collected.

After making the UML diagram, implement the same payment strategy and order status observer problems in Java. Include the test files and exact compilation and execution commands.



1. JavaScript Observer UML diagram

The diagram above represents the JavaScript Observer implementation, including the publisher, observers, order-status service, repository and SMS-provider abstraction.

For your submission, save the diagram in:

```
+sendSms(phone, message) : bool
```

You can render the editable source using Mermaid Live Editor ↗ and export it as PNG or SVG.

3. Next: Start the Java implementations

We will implement the same two patterns in Java, preserving the functional requirements established for Python and JavaScript.

Step 1 — Create the Java directories

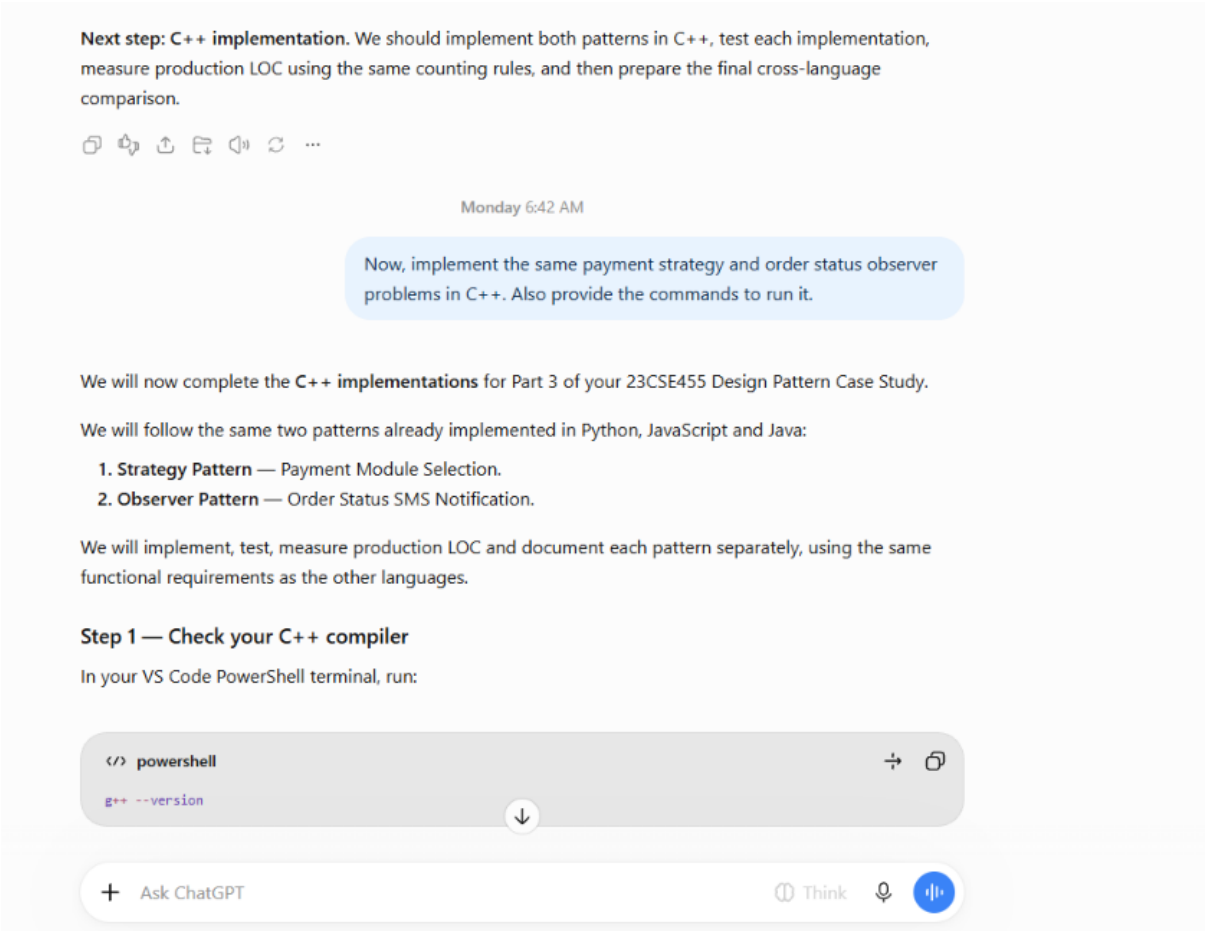
Your current terminal was last in `Observer_Order_SMS\javascript`. In PowerShell, run:

powershell

```
Set-Location ..\..
New-Item -ItemType Directory -Force -Path `
  "Strategy_Payment\java", `
  "Observer_Order_SMS\java"
```

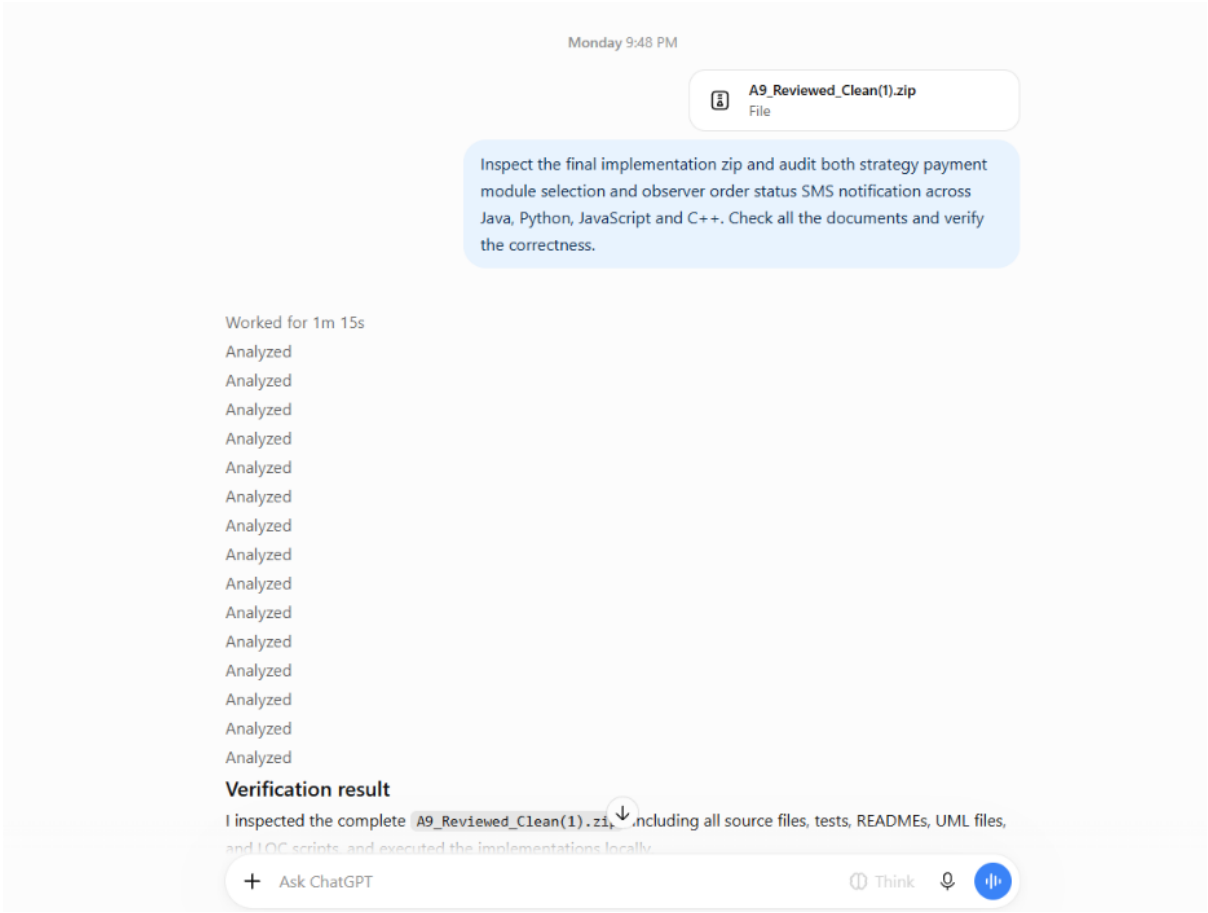
The resulting structure will be:

9. Now, implement the same payment strategy and order status observer problems in C++. Also provide the commands to run it.



10.

Inspect the final implementation zip and audit both strategy payment module selection and observer order status SMS notification across Java, Python, JavaScript and C++. Check all the documents and verify the correctness.



Verification result

I inspected the complete `A9_Reviewed_Clean(1).zip`, including all source files, tests, READMEs, UML files, and LOC scripts, and executed the implementations locally.

Overall verdict

The four-language implementations are functionally correct for both selected patterns.

The two patterns are:

1. **Strategy Pattern — Payment Module Selection**
2. **Observer Pattern — Order Status SMS Notification**

All **8 implementations** compile/run successfully when using the documented/correct commands, and all have passing test suites.

However, the ZIP is **not yet complete as a submission for all 12 expected Part 3 outputs.** The implementation portion is strong, but several required documentation/dataset outputs are absent.

1. Strategy Pattern — Four-language verification

Language	Production LOC	Tests	Result
Java	101	10/10	✅ Pass
Python	80	10/10	✅ Pass
JavaScript	107	10/10	✅ Pass
C++	98	10/10	✅ Pass

Human Verification :

1 Objective

The purpose of human verification was to check whether the code generated with the help of the LLM correctly implemented the selected Strategy and Observer design patterns in Java, Python, JavaScript, and C++. The generated code and documentation were treated as proposed solutions rather than automatically accepted as correct.

I reviewed the source files, executed the documented compilation and test commands, checked the test results, and compared the implementations with the expected behaviour established for both patterns.

2 Verification Procedure

The verification was performed in the following order:

1. Inspected the final implementation ZIP and checked the folder structure, source files, test files, and README instructions.
2. Checked the Strategy Pattern implementation for payment-module selection through a common interface and interchangeable payment strategies.
3. Checked the Observer Pattern implementation for order-status events, listener notification, status filtering, and SMS-provider invocation.
4. Recorded the test results and checked the production lines of code (LOC) using the supplied counting scripts.
5. Reviewed the implementation and documentation for inconsistencies, incorrect execution commands, and unsupported claims.

3 Command Execution and Test Results

The following table records the results reported for the final eight test suites. The commands used must correspond to the actual commands executed in the development environment.

Pattern	Language	Recorded result
Strategy — Payment Module Selection	Java	10/10 passed
Strategy — Payment Module Selection	Python	10/10 passed
Strategy — Payment Module Selection	JavaScript	10/10 passed
Strategy — Payment Module Selection	C++	10/10 passed
Observer — Order Status SMS Notification	Java	10/10 passed
Observer — Order Status SMS Notification	Python	10/10 passed
Observer — Order Status SMS Notification	JavaScript	10/10 passed
Observer — Order Status SMS Notification	C++	10/10 passed
Total	**Eight test suites**	**80/80 passed**

The reported result was 80 passing tests out of 80. Each suite was considered separately so that a successful result for one pattern or language was not treated as proof that the other implementations also worked.

For the final submission, the actual terminal output for each suite should be retained as supporting evidence. The table alone is a summary of the recorded results.

Language-Specific Verification

A. Java

I checked the Java source files, test classes, package declarations, and compilation instructions. I used the appropriate compilation and execution commands for the package structure and reviewed the resulting test output.

The recorded results were 10/10 tests passed for Strategy and 10/10 tests passed for Observer.

B. Python

I checked the Python modules, imports, class interfaces, and test files. I reviewed the execution instructions to ensure that the tests were run from the correct project directory and that the required modules could be imported.

The recorded results were 10/10 tests passed for Strategy and 10/10 tests passed for Observer.

JavaScript

I checked the JavaScript implementation files and test scripts, including the relationship between the exported classes and their imports. I reviewed the documented Node.js execution commands and the test output.

The recorded results were 10/10 tests passed for Strategy and 10/10 tests passed for Observer.

C++

I checked the C++ source and test files, compilation instructions, and the use of the compiler. I also considered the source-file inclusion structure when checking the compilation commands to avoid duplicate definitions.

The recorded results were 10/10 tests passed for Strategy and 10/10 tests passed for Observer.

5 Production LOC Verification

The production LOC values recorded during verification were:

Pattern	Java	Python	JavaScript	C++
Strategy	101	80	107	98
Observer	234	151	201	170

The LOC values were checked against the supplied counting scripts and the production source files. Test code and production code were treated separately to make the cross-language comparison consistent.

These figures should be retained only if they match the final source files and the counting method used for the submission.

6 Manual Review of Design-Pattern Behaviour

Strategy Pattern — Payment Module Selection

The source code was reviewed to check whether payment selection was represented through a common contract and whether alternative payment implementations could be selected without placing all provider-specific behaviour in one central class.

The review focused on the common interface, the individual strategy implementations, the selection mechanism, and the behaviour when a requested strategy was unavailable.

The implementation was considered consistent with the intended Strategy Pattern when the common contract and interchangeable implementations were used as designed.

Observer Pattern — Order Status SMS Notification

The source code was reviewed to check whether order-status changes were represented as events and whether a listener responded to the relevant event without the publisher directly performing all notification work.

The review focused on the event data, publisher, listener, status filtering, and SMS-provider call. It also considered the handling of missing phone numbers and notification failures where these behaviours were implemented.

The implementation was considered consistent with the intended Observer Pattern when the event publisher and listener were separated and the listener handled the relevant status changes.

Four-Language Implementations

1 Objective

The objective of this activity was to implement two design patterns in four programming languages: Java, Python, JavaScript, and C++. The implementations were developed to compare how different languages express the same design principles while preserving the intended behaviour.

The two selected patterns were:

1. **Strategy Pattern – Payment Module Selection**
2. **Observer Pattern – Order Status SMS Notification**

Each implementation was tested independently to verify its functionality and consistency with the intended design.

2 Strategy Pattern – Payment Module Selection

Problem Description

The Strategy Pattern allows a payment operation to be performed using different payment methods without tightly coupling the payment service to a specific implementation. The payment method can be selected at runtime, allowing the system to support multiple payment strategies.

Implementation

The Strategy Pattern was implemented in Java, Python, JavaScript, and C++. Each implementation defines a common abstraction for payment operations and provides concrete implementations for different payment methods, including Card and UPI.

A payment service or context uses the selected strategy to perform the payment operation. This separates payment selection from payment execution and makes it easier to extend the system with additional payment methods.

The implementations cover the following behaviours:

- Selecting a payment method at runtime.
- Supporting Card and UPI payment strategies.
- Handling provider selection without case sensitivity.
- Validating payment requests and rejecting invalid inputs.
- Preventing duplicate registration or selection where applicable.
- Supporting extension through additional payment strategies.

Verification Results

All four implementations were executed and verified against their respective test suites. Each implementation passed 10 tests, giving a total of 40 passing tests for the Strategy Pattern.

Language	Production LOC	Tests passed
Java	101	10/10
Python	80	10/10
JavaScript	107	10/10
C++	98	10/10
Total	**386**	**40/40**

The LOC figures represent production code under the counting rules used for this project, excluding blank lines and comment-only lines.

3 Observer Pattern – Order Status SMS Notification

Problem Description

The Observer Pattern establishes a one-to-many relationship between an event publisher and its registered observers. When the publisher announces an event, the

registered observers are notified without the publisher needing to know their internal implementation details.

For this activity, the pattern was applied to order-status changes. When an order transitions to a relevant status, such as SHIPPED or DELIVERED, registered observers can react to the event.

Implementation

The Observer Pattern was implemented in Java, Python, JavaScript, and C++. Each implementation contains an order-status event publisher and observer components that respond to relevant events.

The implementations demonstrate the following behaviours:

- Registering observers with the event publisher.
- Publishing notifications when valid order-status transitions occur.
- Reacting to SHIPPED and DELIVERED events.
- Separating SMS notification logic from order-status processing.
- Supporting an audit observer alongside the SMS observer.
- Handling missing phone numbers or unavailable SMS-provider functionality.
- Testing the notification behaviour without requiring an actual SMS delivery.

The SMS and audit observers demonstrate how additional reactions can be introduced without placing all notification logic inside the order-processing component.

Verification Results

All four Observer implementations were executed and verified against their respective test suites. Each implementation passed 10 tests, giving a total of 40 passing tests for the Observer Pattern.

Language	Production LOC	Tests passed
Java	234	10/10
Python	151	10/10
JavaScript	201	10/10
C++	170	10/10
Total	**756**	**40/40**

The LOC figures represent production code under the counting rules used for this project, excluding blank lines and comment-only lines.

4 Overall Verification Summary

The verification covered eight implementations: two design patterns in each of four programming languages.

Pattern	Java	Python	JavaScript	C++	Total
Strategy	10/10	10/10	10/10	10/10	40/40
Observer	10/10	10/10	10/10	10/10	40/40
Total	**20/20**	**20/20**	**20/20**	**20/20**	**80/80**

All 80 tests passed across the eight implementations. The results support the functional correctness of the tested scenarios and show that both patterns can be implemented successfully in all four languages.

The test results apply to the standalone implementations and their defined test cases. They do not establish successful integration with a live payment gateway or delivery of real SMS messages.

5 Conclusion

The four-language implementation exercise demonstrated that the Strategy and Observer Patterns can be expressed in Java, Python, JavaScript, and C++ while preserving their core design principles.

The Strategy Pattern separates payment-method selection from payment execution, supporting runtime selection and extensibility. The Observer Pattern separates order-status event publication from the components that react to those events, allowing SMS and audit functionality to be maintained independently.

The implementations also reveal differences in code size and language-specific structure. These differences are examined further in the cross-language comparison using production LOC, implementation structure, complexity, and other available measurements.